

UNIT I

Database

A database is an organised collection of data stored electronically in computer systems.

Databases Management Systems

- A Database Management System (DBMS) is software that enables users to define, create, manage, and manipulate databases.
- It acts as an interface between users/applications and the database.
- DBMS helps in
 - ensuring data integrity and consistency
 - enhancing data sharing
 - ensuring data security
 - provides a centralized view of information across an organization.

SQL (Structured Query Language) is a programming language used for managing and manipulating data in relational databases. It allows you to insert, update, retrieve, and delete data in a database.

DBMS vs File Systems

Problems in traditional file management

Problem	Traditional File System	DBMS Solution	Example
Data Redundancy	In a traditional file system, the same data is stored in multiple files, leading to duplication.	DBMS provides controlled redundancy using centralized database design, solved by normalization and proper schema design.	A student's details stored separately in admission and exam files → duplication.
Data Inconsistency	Multiple copies of the same data may not match after updates.	DBMS ensures consistency by maintaining a single source of truth and using transactions.	Student address updated in one file but not in another.
Difficulty in Accessing Data	New programs must be written for each query, making access inefficient.	DBMS provides SQL for flexible and efficient data retrieval.	To find students with marks > 80, a program must be written in file system.
Data Isolation	Data is scattered in different files and formats, making integration difficult.	DBMS stores data in a unified structure with a common schema.	Student data in one file and marks in another, difficult to combine.
Integrity Problems	No proper enforcement of rules like valid data or relationships.	DBMS enforces integrity using constraints like primary key and foreign key.	Entering invalid age like -5 in file system.
Atomicity Problems	Operations may be partially completed, leading to incorrect data.	DBMS ensures atomicity using transactions (commit or rollback).	Money deducted from one account but not added to another.
Concurrent Access Anomalies	Simultaneous access can cause conflicts and incorrect updates.	DBMS controls concurrency using locking and scheduling.	Two users updating the same bank balance at the same time.
Security Problems	No proper access control or data protection.	DBMS provides authentication, authorization, roles, and privileges.	Any user can access or modify sensitive employee salary data.

Difference between Trad vs Modern data management systems

Aspect	File Processing System	DBMS
Data Redundancy	High data redundancy due to duplication of data in multiple files.	Low data redundancy due to centralized and controlled data storage.
Data Consistency	Data inconsistency is high because multiple copies may not be updated simultaneously.	Ensures data consistency by maintaining a single version of data with constraints.
Data Access	Difficult to access data; requires writing separate programs for each task.	Easy and flexible data access using query languages like SQL.

Aspect	File Processing System	DBMS
Data Isolation	Data is scattered across different files and formats.	Data is integrated and stored in a unified database system.
Data Structure	Data is unstructured and stored in isolated files.	Data is structured and organized with well-defined relationships.
Backup & Recovery	Backup and recovery are manual and error-prone.	Provides automatic backup and recovery mechanisms.
Data Independence	Users need to know physical file details to access data.	Provides data independence; users are not concerned with physical storage.
Data Sharing	Data sharing is difficult between applications.	Data sharing is easy among multiple users and applications.
Concurrency Control	No proper mechanism for handling multiple users simultaneously.	Supports concurrent access with proper control mechanisms like locking.
Security	Limited or no proper security mechanisms.	Provides strong security using authentication, authorization, and access control.

Need of Databases

The purpose of a database system is to overcome the limitations of file-based systems by organizing data efficiently.

Traditional file systems lack:

- flexibility
- lead to duplication
- make it difficult to manage data consistency
- no security

A DBMS provides a unified framework to handle large amounts of data with minimal redundancy and high reliability.

Advantages of databases

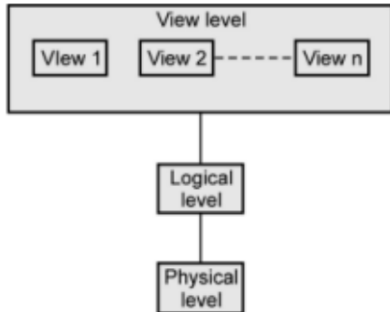
Advantage	Description	Benefit
Data Abstraction	DBMS hides complex data storage details and provides a simplified view using schemas and views.	Users can interact with data without knowing how it is physically stored.
Data Independence	Changes in data structure or storage do not affect application programs (logical and physical independence).	Reduces maintenance effort when database structure changes.
Data Security	DBMS provides authentication, authorization, and access control mechanisms.	Ensures confidentiality and integrity of data.
Data Consistency and Integrity	DBMS enforces constraints like primary key, foreign key, and domain constraints.	Prevents data anomalies and inconsistencies.
Concurrency Control	DBMS allows multiple users to access data simultaneously using locking and transactions.	Ensures correct results in multi-user environments.
Backup and Recovery	DBMS provides mechanisms to backup and restore data after failures.	Prevents data loss and ensures system reliability.

Database-System Applications

- Databases are used in almost every sector today. They serve as the backbone of many applications that require storage, retrieval, and analysis of data.
- From small businesses to large enterprises, databases power daily operations.
 1. Banking Systems: Manage accounts, customer info, loans, and transactions.
 2. Airlines: Handle schedules, bookings, and ticketing.
 3. Educational Institutes: Manage student records, marks, fees, and courses.
 4. E-commerce Platforms: Store product catalogs, orders, payments, and customer data.
 5. Healthcare Systems: Maintain patient records, prescriptions, and treatment histories.

View of Data

A DBMS provides data abstraction through three levels of views. This abstraction helps different users interact with the database without worrying about the complexities of data storage. It hides certain details of how the data are stored and maintained.



There are three levels of abstraction in DBMS:

1. Physical Level
2. Logical Level
3. View (External) Level

- **Physical Level:** Describes how data is stored physically on disks, using file systems, indexes, and storage blocks.

Example:

Data stored in files, blocks, indexes on disk.

- **Logical Level:** Describes what data is stored and the logical relationships among data. It hides the physical details from users. It is the view of the entire database schema.

Example:

The schema of an Employee table might be:

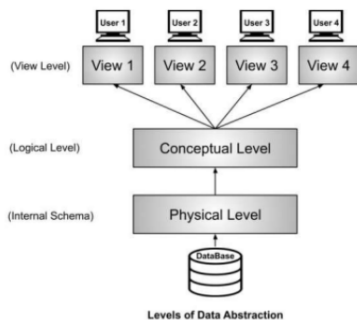
Employee(EmpID, Name, Department, Salary)

- **View Level:** Describes only part of the database relevant to a particular user. Different users can have different views of the same data. Hides unnecessary data for security and simplicity.

Example:

A student sees only marks and attendance, not salary details of faculty.

Data Abstraction



- Data Abstraction in a Database Management System (DBMS) is the technique of hiding complex internal details of data storage and showing only the required information to users.
- It improves simplicity, security, and data independence.
 - Security: Different users get different views (e.g., HR vs. Intern).
 - Simplicity: Users are not bothered with how data is stored.
 - Maintainability: Physical changes don't affect user applications.
 - Flexibility: Logical design can be changed without altering views.

Data independence

- Data independence is an ability by which one can change the data at one level without affecting the data at another level. Here level can be physical, conceptual or external.
- By this property, the structure of the database or the values stored in the database can be easily modified by without changing the application programs.
- Types of Data Independence

1. Physical Data Independence

Physical Data Independence refers to the ability to change the physical storage of data without affecting the logical (conceptual) schema or application programs.

Example: Changing data storage from HDD to SSD without modifying table definitions or applications.

1. Logical Data Independence

Logical Data Independence refers to the ability to change the logical structure of the database without affecting the external views or application programs.

Example: Adding a new column Email to the STUDENT table without affecting existing applications.

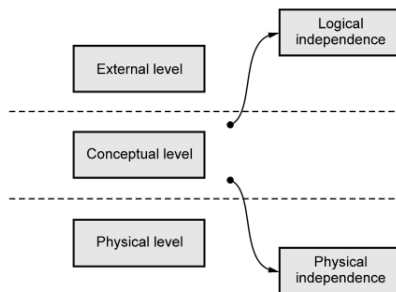


Fig. 1.10.1 Data independence

Aspect	Physical Data Independence	Logical Data Independence
Definition	Ability to change the internal schema without affecting the conceptual schema.	Ability to change the conceptual schema without affecting the external schema (user view).
Level Affected	Internal → Conceptual level	Conceptual → External level
Type of Change	Changes in storage structure, file organization, indexing methods, etc.	Changes in table structure, attributes, relationships, etc.
Complexity	Easier to achieve	More difficult to achieve
User Impact	No impact on application programs or users	No impact on user views, but harder to maintain
Examples	Changing indexing method or storage location of data	Adding/removing attributes or modifying relationships in schema

DBMS Architecture

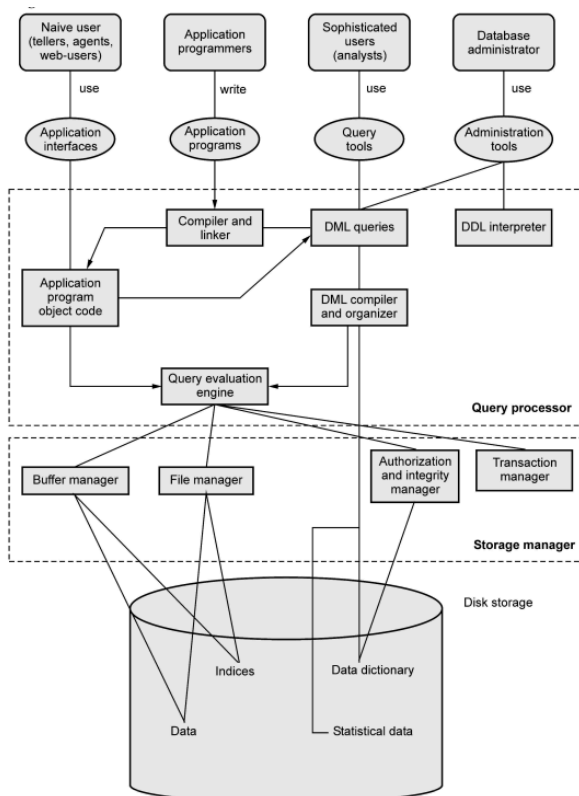


Fig. 1.8.2 Architecture of database

- A Database Management System (DBMS) is structured into different layers to efficiently manage data, process queries, and ensure security and consistency.
- The architecture in the diagram is broadly divided into **four levels**: Users, Query Processor, Storage Manager, and Disk Storage.

1. Different users interact with the DBMS in different ways:

- **Naïve Users:** These users do not know DBMS or SQL. They just use ready-made interfaces like forms or apps.

Example: A student filling an online college form, a customer using an ATM, or booking tickets on a website.

- **Application Programmers:** They write programs that interact with the database using languages like Java, Python, etc.

Example: A developer creating a banking app or an e-commerce website that stores user data in a database.

- **Sophisticated Users (Analysts):** They understand databases and directly use query languages (like SQL) or tools to get data, but don't build full applications.

Example: A data analyst running SQL queries to find sales trends or generating reports using tools like Power BI.

- **Specialized users :** They create special-purpose applications that use databases but are not typical business apps.

Scientists using databases for research simulations, AI/ML systems, CAD (Computer-Aided Design) software.

- **Database Administrator (DBA):** Uses administration tools to manage the database.

These users provide input in the form of **queries or programs**.

2. The query processor interprets and translates user queries into a form that the DBMS can execute efficiently It consists of:

- **DDL Interpreter:** Processes Data Definition Language commands (like CREATE, ALTER) and updates the data dictionary.
- **DML Compiler and Organizer:** Converts DML queries (SELECT, INSERT, etc.) into low-level instructions.
- **Query Evaluation Engine:** Executes the query efficiently by interacting with the storage manager.
- **Compiler and Linker:** Converts application programs into executable form.

This layer ensures that user queries are translated into an efficient execution plan.

3. Storage manager is responsible for managing **physical storage of data** in the database. It acts as an interface between the query processor and disk storage.

Components include:

- **Buffer Manager:** Handles data transfer between main memory and disk.
- **File Manager:** Handles file organization and access methods.
- **Authorization and Integrity Manager:** Ensures access control and maintains data integrity constraints.
- **Transaction Manager:** Maintains ACID Properties

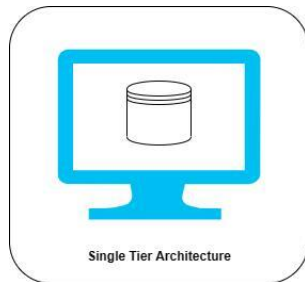
These components ensure proper data storage, retrieval, and security.

4. This is the physical storage where actual data resides. It includes:

- **Data Files:** Store actual database records.
- **Indices:** Provide fast access to data.
- **Data Dictionary (Metadata):** Stores schema and structure of the database.
- **Statistical Data:** Used by the query optimizer to improve performance.

DBMS Architecture: 1-level, 2-Level, 3-Level

1. 1-Tier Architecture (Single-Level / Centralized)



In this architecture, the **user interface, application logic, and database** are all present on the **same system**.

- The user directly interacts with the database without any network.
- There is no separation between client and server.
- Mainly used for **local applications and development/testing purposes**.

Example: Microsoft Access

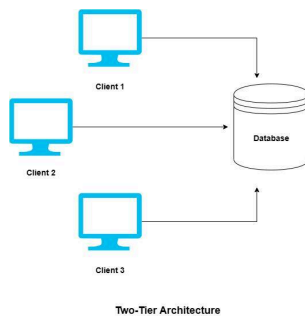
Advantages:

- Simple and easy to use
- No network dependency

Disadvantages:

- Poor security
- Not suitable for large or multi-user systems

2. 2-Tier Architecture (Client-Server)



This architecture divides the system into **client and server**.

- The **client (application)** sends requests directly to the **database server**.
- Communication is done using APIs like **ODBC (Open Database Connectivity)** and **JDBC (Java Database Connectivity)**.
- The database server processes queries and sends results back to the client.

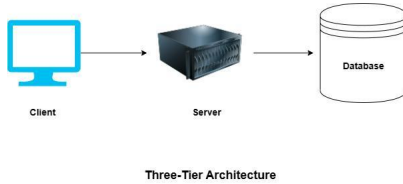
Advantages:

- Better performance than 3-tier for small systems
- Simple structure

Disadvantages:

- Limited scalability
- Less secure (client directly accesses DB)

3. 3-Tier Architecture



This architecture introduces an **intermediate layer (application server)** between client and database.

- **Client → Application Server → Database Server**
- The client interacts only with the application server.
- The application server handles **business logic, validation, and processing**.
- The database server manages **data storage and transactions**.

Advantages:

- High security (no direct DB access)
- Better scalability and performance
- Suitable for **large web applications**

Disadvantages:

- More complex
- Higher development and maintenance cost

Data Models

- Data models are conceptual tools that define how data is organized, stored, and accessed in a DBMS.
- They specify relationships among data elements and enforce constraints to ensure data integrity.
- They act as a blueprint for designing and implementing a database system.
- Types of Data Models:
 - Relational Data Model

The Relational Data Model, proposed by E. F. Codd, organizes data into tables (relations) consisting of rows (tuples) and columns (attributes).

 - Key Concepts
 - Relation (Table): Collection of related data
 - Tuple (Row): Single record in a table
 - Attribute (Column): Field describing data
 - Primary Key: Uniquely identifies each record
 - Foreign Key: Establishes relationship between tables
 - Entity-Relationship (ER) Model: Uses ER diagrams to show entities, attributes, and relationships visually.
 - Object-Based Model: Based on object-oriented programming; supports classes, objects, and inheritance.
 - Hierarchical Model: Data is stored in a tree structure. Each parent can have many children, but each child has only one parent. This model represents the relationship in 1 : N types.
 - Network Model: Similar to the hierarchical model but allows many-to-many relationships using graph-like structures. It helps to address M : N relationship.
 - Semi-structured Model: Data doesn't follow a rigid schema, used in XML and JSON formats.

ER Model

The **Entity-Relationship (ER) Model** is a conceptual data model used to design and represent the structure of a database. It helps in identifying entities to be represented in the database and how those entities are related.

The ER model consists of three basic concepts –

1. Entities :An **entity** is an object that exists and is distinct from other objects.
2. Relationships : A **relationship** represents an association between two or more entities.
3. Attributes : An **attribute** is a property or characteristic that describes an entity.

ER Diagram Attributes:

Symbol	Meaning
	Entity Type
	Weak Entity Type
	Relationship Type
	Identifying Relationship (for weak entity)
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E ₂ in R
	One to Many relationship between entity sets E ₁ to E ₂

Strong Entity Set:

A strong entity set is an entity that has a primary key of its own and can be uniquely identified independently.

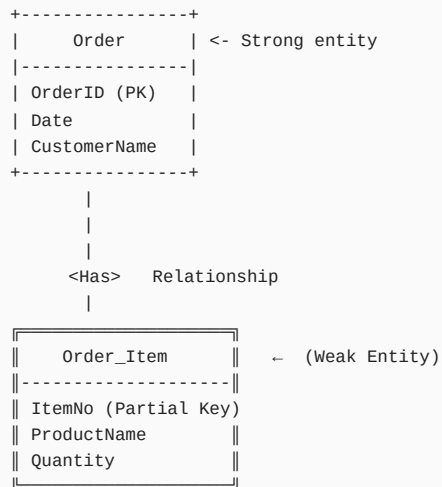
Weak Entity Set:

A weak entity set is an entity that does not have a primary key of its own and depends on a strong entity set for identification.

Strong Entity Set	Weak Entity Set
Has its own primary key.	Does not have sufficient attributes to form a primary key.
Represented by a single rectangle in ER diagram.	Represented by a double rectangle.
Primary key is shown by a solid underline.	Partial key (discriminator) is shown by a dashed underline.
Exists independently of other entities.	Depends on a strong entity for existence.
Also called a dominant entity set.	Also called a subordinate entity set.
Relationship between strong entities is shown by a single diamond.	Identifying relationship is shown by a double diamond.
Primary key uniquely identifies each entity.	Primary key is a combination of partial key and primary key of the strong entity.

Example:

- **Strong Entity: Order** (OrderID as primary key)
- **Weak Entity: Order_Item**
- Each **Order** has a unique **OrderID**, so it is a **strong entity**.
- **Order_Item** (items inside an order) cannot be uniquely identified on its own.
- It depends on **Order** for identification.
- Weak Entity identified using (**OrderID + ItemNo**) → composite key



Types of Attributes:

- **Simple Attribute:** Cannot be divided further (e.g., Age).
- **Composite Attribute:** Can be broken down into sub-parts (e.g., Name → First Name, Last Name).
- **Single-valued Attribute:** Contains only one value (e.g., Roll Number).
- **Multivalued Attribute:** Can have multiple values (e.g., Phone Numbers).
- **Derived Attribute:** Computed from other attributes (e.g., Age from Date of Birth).

Cardinality Ratio

The **cardinality ratio** defines how many instances of one entity can be associated with instances of another entity.

- **One-to-One (1:1):** Each entity in one set is related to at most one entity in the other set and vice versa.
 - One student has one ID card.
- **One-to-Many (1:N):** One entity in the first set is related to many entities in the second set, but each entity in the second set is related to only one entity in the first.
 - One teacher teaches many students.
- **Many-to-Many (M:N):** Entities in both sets can be related to many entities in the other set.
 - Many students enroll in many courses.

Keys:

Keys are used to identify and access records in a database table efficiently.

Example Table:

Roll_No	StudentID	Name	Email	Dept
101	S001	Rahul	rahul@gmail.com	CS
102	S002	Neha	neha@gmail.com	IT
103	S003	Amit	amit@gmail.com	CS

Types of keys:

1. **Primary Key:** A unique identifier for each record in a table, ensuring data integrity and fast data retrieval.

- PRIMARY KEY (Roll_No)

2. **Foreign Key:** A foreign key is a primary key from one table that is used to establish a relationship with another table enforcing referential integrity and maintaining data consistency across relations

- FOREIGN KEY (Dept) REFERENCES Department(DeptName)

3. **Super Key:** A combination of columns(attributes) that can uniquely identify records, within a RDBMS.

- Roll_No
- StudentID
- Email
- (Roll_No, StudentID)
- (Roll_No, Email)
- (StudentID, Email)
- (Roll_No, Name)
- (Roll_No, StudentID, Name)
- (Roll_No, StudentID, Email)
- (Roll_No, StudentID, Name, Email)
- (Roll_No, StudentID, Name, Email, Dept)

4. **Composite Key:** A primary key with combination of two or more columns that, together, uniquely identify each record in a table. Composite keys are used when a single attribute is not sufficient to uniquely identify a record in a table.

- (Roll_No, StudentID)
- (Roll_No, Email)
- (StudentID, Email)
- (Roll_No, StudentID, Name)

5. **Candidate Key:** A potential primary key candidate that uses smallest set of attributes without any redundant ones that uniquely identifies records in a table but is not currently designated as the primary key. This means that a candidate key cannot have duplicate values, and it cannot contain any null values.

- Roll_No
- StudentID
- Email

6. **Alternate Key:** Another candidate key that is not chosen as the primary key.

- StudentID
- Email

7. **Surrogate Key:** When a table does not have a Primary or Composite key, a system-generated unique identifier is used as the primary key when a natural primary key is not available or is not suitable for performance or security reasons.

ID	Roll_No	Name	Email
1	101	Rahul	rahul@gmail.com

8. **Unique Key:** Ensures that each value in the specified column is unique, but unlike the primary key, a table can have multiple unique keys. **Unique keys can contain null values.**

- email

Database Design Process

Database design must follow a step-by-step method to ensure the system meets user requirements and functions efficiently.

1. Requirement Analysis

Identify user needs, data, and business rules.

2. Conceptual Design

Create ER diagram with:

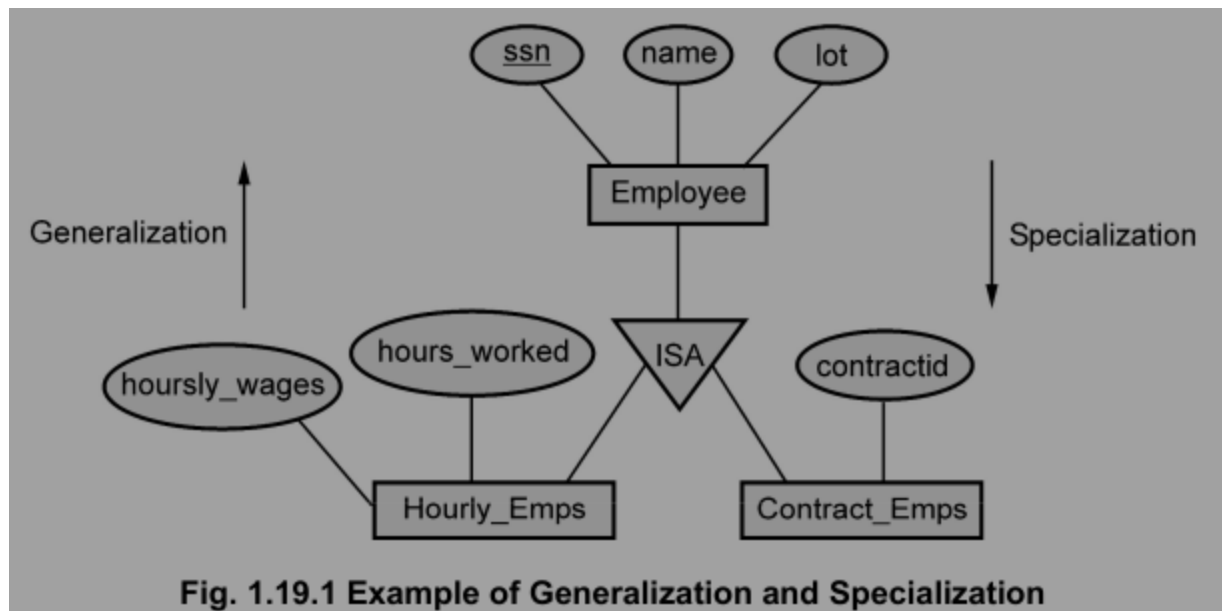
- Entities: Student, Course, Instructor

- Attributes
 - Relationships:
 - Enrolled_in (M:N)
 - Taught_by (1:N)
3. **Logical Design**
Convert ER to tables:
- Student(StudentID, Name, Dept)
 - Course(CourseID, CourseName, Credits, InstructorID)
 - Instructor(InstructorID, Name, Dept)
 - Enrollment(StudentID, CourseID, Grade)
 - PK: (StudentID, CourseID)
4. **Schema Refinement**
Apply normalization (1NF–3NF) to remove redundancy and anomalies.
5. **Physical Design**
Define storage, indexing, and access methods.
6. **Security Design**
Define roles, permissions, and access control.
-

Extended ER Features (EER)

EER enhances the basic ER model to represent more complex relationships such as hierarchy and shared relationships.

- Specialization: Specialization is a top-down approach where a higher-level entity is divided into lower-level subentities based on distinguishing characteristics.
- Generalization: Generalization is the opposite of specialization. It is a bottom-up approach where multiple entities with similar characteristics are combined into a higher-level entity.
- Concepts in Specialization and Generalization:
 - Superclass : An entity type that represents a general concept at a high level, is called superclass.
 - Subclass : An entity type that represents a specific concept at lower levels, is called subclass.
- The symbol used for specialization/ Generalization is:
ISA



- Aggregation: Standard ER models cannot directly represent relationships among relationships. Aggregation solves this limitation by treating a relationship as an entity for participating in other relationships. Useful for modeling complex scenarios.

